Developer Build Brief

I Gave Claude $20,000 To Trade Crypto

Seven-Day Paper Trading / Blind Replay System

Purpose

Build a real, dashboard-backed Claude crypto trading system that can produce a seven-day paper portfolio challenge for YouTube. The first priority is a batchable video workflow: Miles can react cold to accurate snapshots, trade logs, and results in one filming session. The second priority is that the system is genuinely functional enough to teach users how this could be built and later connected to real paper trading or exchange testnets.

Field	Decision
Video title	I Gave Claude $20,000 To Trade Crypto
Money type	$20,000 paper portfolio. No real money for this first build unless explicitly changed later.
Default mode	Seven-day historical replay OR seven-day paper-forward run, selected in config. Language in the video must match the mode.
Filming goal	Miles reacts cold in one batch shoot using prepared daily snapshots and a final reveal package.
Core promise	Real system, accurate data, locked rules, no fabricated PnL.

1. Build Philosophy

This should not be a fake dashboard or a manually edited PnL story. The system should be functional at the architecture level: it ingests market data, builds a market packet, asks Claude for structured trading decisions, validates those decisions against risk rules, executes them in a paper portfolio engine, and produces repeatable logs, charts, and snapshots. The video can be cinematic and batch-filmed, but the underlying run should be real.

North star

Batch the filming, preserve the uncertainty, and keep the system real enough that viewers can understand how to build a version themselves.

Principle	Implementation Standard
Paper, not real money	All results must be labeled as paper trading or simulation unless a future episode genuinely uses real capital.
Accurate, not perfect	Use clean OHLCV data, clear execution rules, fees, slippage assumptions, and reproducible calculations.
Functional, not cosmetic	The dashboard should be generated from logs and portfolio state, not manually designed to fit a story.
Story-friendly, not outcome-rigged	The developer can prepare key moments after the run, but cannot change the window, prompt, or rules after seeing the result.
Educational	The final output should show enough of the pipeline that users understand data -> Claude -> risk checks -> paper execution -> dashboard.

2. Recommended First Video Format

For the first video, use a seven-day challenge. The viewer-facing story is that Claude receives a $20,000 paper portfolio and trades crypto for seven days. Miles has not seen the results before the reaction shoot. The editor can present the reveal day by day, with location changes and full-screen reactions, but the framing must match the mode used.

Mode	What Happens	Correct Viewer Language
Historical replay	The system runs Claude through one historical seven-day market window. This is easiest for immediate production.	"We ran Claude through one week of historical crypto market data with a $20,000 paper portfolio."
Paper-forward run	The system runs for seven actual days in paper mode using live market data, then Miles reacts after it finishes.	"We let Claude trade a $20,000 paper portfolio for one week, and I am reacting to the results now."
Future live real-money run	A later premium episode where real funds are connected and tracked in real time.	"This time it is real money and live market execution." Only use this if true.

Important wording rule

If the first build is a historical replay, do not call it live trading. If it is a paper-forward run, it is fine to say it ran for a week, but still call it paper trading. In both cases, Miles can truthfully say he has not seen the result before reacting.

3. System Architecture

The build should be modular. The first version can be simple, but the architecture should make it obvious how the system could later connect to real exchange testnets, live paper trading, and eventually real execution if approved.

Module	Purpose	Must Produce
Config Manager	Stores all episode parameters before the run starts.	Run ID, mode, dates, assets, prompt version, risk rules, benchmark list.
Market Data Ingestor	Pulls historical or live OHLCV data and normalizes it.	Clean candles, timestamps, current prices, indicator inputs.
Indicator Engine	Computes signals Claude can use without future leakage.	Returns, trend, volatility, moving averages, drawdowns, RSI or momentum if used.
Claude Decision Agent	Receives the market packet and outputs structured portfolio decisions.	JSON decision, reasoning, strategy choice, target allocations, risk note.
Risk Validator	Checks Claude output against hard portfolio constraints.	Approved/rejected actions, corrected allocation if needed, validation notes.
Paper Portfolio Engine	Executes validated trades with fees/slippage assumptions.	Positions, cash, trades, PnL, drawdown, equity curve.
Dashboard + Snapshot Generator	Turns logs into visual screens for filming and editing.	Daily snapshots, charts, trade log, scorecard, export bundle.

```
Market data -> Indicator engine -> Claude decision packet -> Claude JSON decision
-> Risk validator -> Paper execution engine -> Portfolio state
-> Daily snapshot dashboard -> Export bundle for Miles + editor
```

4. Dashboard Requirements

The dashboard is the key production artifact. It should let Miles react through the challenge as if he is watching a match unfold. It needs to be visually clean enough to screen-record, screenshot, and hand to the editor without manual reconstruction.

Screen / Component	Required Content	Why It Matters
Overview page	Starting capital, current portfolio value, total PnL, percentage return, cash, crypto exposure, max drawdown, number of trades, best/worst trade.	This is the main reveal screen and can be used in the intro, midpoint, and final scorecard.
Seven-day timeline	Day 0 setup plus Day 1 through Day 7 snapshots. Each day should be clickable/exportable.	Gives the editor clean story beats and lets Miles react day by day.
Daily snapshot page	Portfolio value, daily PnL, positions, trades executed, Claude reasoning, selected strategy,	This is the core content for the reaction shoot.

	market context, benchmark comparison.	
Trade log	Timestamp, asset, buy/sell/hold/rebalance, price, quantity, notional size, fee, slippage, reason, portfolio value after trade.	Proves the run was generated from real decisions, not manually written after the fact.
Claude reasoning panel	Raw prompt input summary and Claude output for that decision point, including risk note and confidence.	Makes the bot feel real and helps viewers learn how to build their own.
Strategy selector panel	Which strategy Claude selected or which preset strategy was active, plus the reason for selection.	Adds a game mechanic and makes the episode easier to narrate.
Benchmark chart	Claude portfolio vs BTC hold, ETH hold, 50/50 BTC/ETH, equal-weight basket, and cash.	Keeps the result honest. Claude may lose money but still outperform a benchmark, or vice versa.
Export controls	Export PNG snapshots, CSV trade log, JSON run log, and a PDF/HTML summary.	Needed for filming, editing, QA, and future repurposing.

Snapshot rule

Every daily snapshot should be generated from the run logs. Do not manually edit screenshots to create a cleaner story. If the dashboard needs a cleaner graphic, create it programmatically from the same data.

5. Strategy Selector and Bot Logic

The first version should not try to create a perfect trading bot. It should create a believable, understandable Claude-driven portfolio manager with a fixed strategy menu. Claude can select a strategy each day or be assigned one strategy for the full run, depending on how complex the developer wants the first build to be. The important part is that the strategy logic and risk constraints are real, explainable, and consistent.

Strategy Option	Description	Claude Should Use When
Trend-following	Favor assets in an uptrend and reduce exposure when trend weakens.	Market structure is strong and higher time-frame momentum is positive.
Momentum rotation	Rotate toward the strongest liquid assets over the recent lookback window.	Altcoins are outperforming and dispersion is high.
Mean reversion	Buy assets that are oversold relative to recent volatility, with strict sizing.	Market is range-bound or a high-quality asset has pulled back sharply.
Breakout	Enter or increase exposure after price clears a recent range/high.	Volatility expands after consolidation.
Risk-off preservation	Move partially or mostly to cash/stables when downside risk is high.	Trend, volatility, or drawdown signals indicate unstable conditions.

Balanced allocation	Hold a diversified basket with moderate exposure.	Signals are mixed and Claude lacks conviction.

Recommended first-video logic: allow Claude to choose one of the above strategy labels at each daily decision point, then output target allocations. The system should validate allocations against hard risk limits. This gives the video a clear narrative: "Claude switched from momentum to risk-off on Day 3," or "Claude ignored the drawdown and stayed in SOL."

Risk Rule	Recommended Default
Starting portfolio	$20,000 paper capital
Asset universe	BTC, ETH, SOL, BNB, XRP, DOGE, LINK, AVAX. Keep episode one to liquid majors.
Decision frequency	Once per day for the first build. Optionally support 4H decisions later.
Max allocation per asset	35% to 40% of portfolio value.
Max total crypto exposure	80% to 100%, configurable. For episode one, 80% is cleaner.
Cash/stable allocation	Allowed. Cash should be treated as USDT/USDC paper balance or plain USD cash.
Leverage	Disabled for episode one.
Shorting	Disabled for episode one.
Fees	Default 0.10% per trade, configurable.
Slippage	Default 0.05% to 0.10% per trade, configurable.

6. Claude Prompt and Output Schema

Claude should receive a structured market packet at each decision point. It should not receive future data. The output should be strict JSON so the paper portfolio engine can parse and execute decisions reliably.

```
Daily market packet should include:
- Run ID and day number
- Current timestamp
- Current portfolio value, cash, positions, and unrealized PnL
- Asset universe with current price, 24h/7d return, volatility, trend status, drawdown
- Recent portfolio decisions and Claude's prior reasoning
- Allowed actions and hard risk constraints
- Benchmark performance so far, if appropriate
- Instruction to output valid JSON only
```

```
{
  "day": 3,
  "market_view": "bullish | bearish | neutral | mixed",
  "selected_strategy": "trend_following | momentum_rotation | mean_reversion | breakout | risk_off | balanced",
  "portfolio_action": "hold | rebalance | de_risk | increase_exposure",
  "target_allocations": {
    "BTC": 0.30,
    "ETH": 0.20,
    "SOL": 0.10,
    "BNB": 0.00,
    "XRP": 0.00,
    "DOGE": 0.00,
```

```
    "LINK": 0.00,
    "AVAX": 0.00,
    "CASH": 0.40
  },
  "reasoning": "Short explanation for the trade decision.",
  "risk_note": "Main risk Claude is managing.",
  "confidence": 0.62
}
```


Validation rule

If Claude outputs invalid JSON, allocations that do not sum to 1.0, an asset outside the universe, leverage, or a position above the cap, the system should reject or normalize the decision and log the correction visibly.

7. Data Accuracy and Anti-Leakage Requirements

The most important technical requirement is that the test is defensible. Claude should only see information that would have existed at each decision point. The dashboard should make it easy to verify exactly what Claude saw, what it decided, and how the portfolio changed.

Requirement	Implementation Detail
No future leakage	At Day N, Claude can only receive market data up to the decision timestamp. Indicators must be calculated using past candles only.
Execution price rule	Define clearly: trades execute at next candle open, same-day close, or another consistent rule. Prefer next daily open for simplicity.
Timezone standard	Use UTC timestamps unless explicitly changed. Display dates consistently on the dashboard.
Source logging	Log the market data source, download timestamp, symbol mapping, and candle interval.
Run reproducibility	Every run should have a run_id, config file, prompt version, model name/version where available, and exported logs.
Fees and slippage	Apply fee/slippage assumptions to every executed trade, not just final PnL.
Benchmark fairness	Benchmarks should use the same start and end timestamps and comparable fee assumptions where relevant.
No cherry-pick edits	After the locked run is generated, do not change the rules or window to create a more interesting PnL curve.

8. Episode Configuration File

The developer should make the whole run configurable from a single file. This makes it easier to reuse the same system for Claude vs ChatGPT, meme coins, AI stocks, leverage, or live paper trading later.

```
episode:
  title: "I Gave Claude $20,000 To Trade Crypto"
```

```yaml
  run_mode: "historical_replay"   # historical_replay or paper_forward
  starting_capital: 20000
  duration_days: 7
  decision_frequency: "1d"
  timezone: "UTC"

assets:
  universe: ["BTC", "ETH", "SOL", "BNB", "XRP", "DOGE", "LINK", "AVAX"]
  quote_currency: "USD"

risk:
  max_asset_allocation: 0.40
  max_total_crypto_exposure: 0.80
  leverage_enabled: false
  shorting_enabled: false
  trading_fee_bps: 10
  slippage_bps: 5

benchmarks:
  - "BTC_hold"
  - "ETH_hold"
  - "BTC_ETH_50_50"
  - "equal_weight_universe"
  - "cash"

exports:
  png_snapshots: true
  csv_trade_log: true
  json_run_log: true
  html_dashboard: true
  summary_pdf: true
```

9. Export Package for Miles and Editor

The developer should deliver a clean folder that Miles can open before filming and the editor can use after filming. The goal is to make the reaction shoot easy and make the edit visually strong without needing to reverse-engineer the data.

Deliverable	Format	Content
Executive summary	PDF or HTML	Starting capital, ending capital, total return, max drawdown, number of trades, benchmark comparison, final verdict.
Daily snapshots	PNG + dashboard pages	Day 0 through Day 7 screens with PnL, positions, trades, reasoning, selected strategy, and key chart.
Trade log	CSV + dashboard table	Every trade with timestamp, price, size, fee, slippage, reason, and portfolio value after execution.
Run log	JSON	All market packets, Claude decisions, risk validation, execution events, and portfolio states.

Charts	PNG/SVG	Portfolio value vs benchmarks, asset allocation over time, drawdown chart, daily PnL bars.
Key moments list	Markdown or PDF	Five to eight moments for Miles to react to: big trade, mistake, pivot, drawdown, recovery, final reveal.
Screen recording	MP4 optional	A guided scroll through the dashboard for easy use in the edit.

Editor note

Do not hand the editor only raw spreadsheets. The dashboard should generate visual, high-contrast, screen-ready moments: "Day 3: Claude rotates to cash," "Largest drawdown," "Final PnL vs BTC," and "Would we trust it?"

10. User Education Layer

A major purpose of the series is to show that users can build their own version. The system should therefore expose the pipeline clearly rather than hiding everything in a black box. The video can briefly show the architecture and then point to future tutorials or resources.

Educational Moment	What To Show
How the bot gets data	Show OHLCV candles, indicators, and the market packet that Claude receives.
How Claude makes decisions	Show the structured JSON output and selected strategy.
How risk rules protect the portfolio	Show a validation layer rejecting invalid allocations or preventing overexposure.
How paper trading works	Show the execution engine converting target allocations into paper trades.
How this could become live	Show that the same architecture could later connect to an exchange testnet or paper-trading broker before real money is considered.

Recommended on-camera explanation: "This is paper money, but the system is not fake. It is the same basic pipeline you would use for a real bot: data comes in, Claude makes a decision, risk rules check it, a paper engine executes it, and the dashboard tracks the result."

11. Future Live / Connected Version

The first build should be architected so the next level is obvious. Do not overbuild live execution now, but keep the interfaces clean enough that live paper trading or exchange testnet trading can be added later.

Phase	Description	Build Now?
Phase 1: Historical replay	Immediate production mode. Run Claude through historical seven-day windows and export snapshots.	Yes.
Phase 2: Paper-forward	Let Claude run for seven actual days against live market data in a paper portfolio. Miles reacts after	Optional but useful.

	the week ends.	
Phase 3: Exchange testnet	Connect the same decision engine to an exchange testnet or sandbox environment for simulated orders.	Design for it, do not prioritize unless easy.
Phase 4: Real money	Only after legal/risk review, with tiny size, strict caps, and clear viewer disclosure.	No for first build.

12. Recommended Build Stack

The developer can choose the stack, but the build should optimize for speed, reliability, and easy visual output. A simple Python backend plus a lightweight web dashboard is enough for v1.

Layer	Suggested Approach
Backend	Python for data ingestion, indicators, paper portfolio engine, logging, and exports.
LLM interface	LLM API client wrapper with prompt templates and structured JSON parsing.
Data storage	Local SQLite, Postgres, or flat JSON/CSV for v1. The key is reproducibility, not scale.
Dashboard	Streamlit, FastAPI + React, Next.js, or any quick web UI the developer can ship cleanly.
Charts	Plotly/Recharts/Matplotlib or dashboard-native charts. Must export PNG/SVG.
Exchange/paper interface	Abstract interface now: execute_order(), get_positions(), get_cash(), get_prices(). Later map this to a testnet or broker API.

13. Acceptance Criteria

The build is ready for filming only when all of the following are true.

- A full seven-day run can be generated from a locked config without manual data editing.
- Each Claude decision is saved as raw output and parsed JSON.
- The risk validator logs approvals, rejections, and corrections.
- The paper portfolio engine calculates trades, fees, slippage, positions, cash, PnL, and drawdown.
- The dashboard shows Day 0 through Day 7 snapshots and can export each as PNG.
- The final report compares Claude against BTC, ETH, 50/50 BTC/ETH, equal-weight basket, and cash.
- The developer can explain exactly what data Claude saw at each decision point.
- Miles can open one dashboard/report and react cold without needing to understand backend files.
- The export bundle includes summary, charts, trade log, run log, and key moments list.
- The result has not been manually changed to improve the story.

14. Developer Task List

Priority	Task	Definition of Done
P0	Create config-driven run system	One config file controls capital, assets, dates/mode, risk rules, benchmarks, and exports.

P0	Build market data + indicator pipeline	System can load seven days of data and create daily Claude packets without future leakage.
P0	Build Claude decision wrapper	Claude returns strict JSON decisions that are stored and parsed.
P0	Build paper portfolio engine	Target allocations become paper trades with fees, slippage, positions, cash, and PnL.
P0	Build risk validator	Invalid allocations, leverage, shorting, and oversized positions are rejected or corrected.
P0	Build dashboard snapshots	Day 0 through Day 7 pages are screen-ready and exportable.
P1	Build final report exporter	Summary PDF/HTML, charts, trade log, JSON run log, and key moments list.
P1	Add benchmark comparisons	BTC, ETH, 50/50 BTC/ETH, equal-weight universe, and cash.
P2	Add paper-forward mode	Same engine can run once per day for seven actual days against live prices.
P2	Add exchange/testnet adapter interface	Execution interface is ready for future testnet or live connector.

15. Filming Workflow

The finished workflow should let Miles batch the entire video in one session while still reacting genuinely. The developer should not spoil the result before filming.

1. Developer locks the config and runs the system.
2. Developer checks for technical errors only. Do not change the outcome because it is boring or negative.
3. Developer prepares the dashboard, export bundle, and key moments list.
4. Miles receives the dashboard during filming and reacts day by day.
5. Editor uses snapshots, charts, screen recordings, and Miles reactions to build the story.
6. Final video includes clear paper-trading language and explains that the system is functional, not a fake PnL graphic.

Best intro wording

"I gave Claude a $20,000 paper portfolio and let it trade crypto for seven days. My developer handled the backend, logged every trade, and I have not seen the results yet. We are going to reveal what happened day by day."

16. Do Not Build / Do Not Do

- Do not build fake PnL screenshots that are not generated from the paper trading engine.
- Do not leak future price data into Claude prompts or indicators.
- Do not rerun the same prompt repeatedly until the outcome is more dramatic, unless that is disclosed as experimentation and not the final locked test.

- Do not manually change trades, prices, or allocations after the run.
- Do not call a historical replay live trading.
- Do not imply real money was used unless a future episode actually uses real money.

17. One-Line Build Summary

Developer instruction

Build a real, config-driven Claude crypto paper-trading system where Claude manages a $20,000 paper portfolio over seven days, with locked rules, accurate market data, risk validation, paper execution, dashboard snapshots, trade logs, benchmark comparisons, and an export package Miles can react to cold in one filming session.